Name : Neeraj Pandey
Batch : T21
Roll no : 2401073

Experiment 2: To Containerize Application using Docker

Aim:

To containerize application using Docker

LO4: Containerize, orchestrate and deploy applications using Docker, Docker Compose, Kubernetes and cloud infrastructure tools.

Theory

Steps to Install Docker

1. Visit the official Docker website and download Docker Desktop for your operating system (Windows).
2. Locate the downloaded installer file and double-click it to start the installation.
3. Follow the installation wizard. When prompted, select the option to Use WSL 2 instead of Hyper-V, as it provides better performance on Windows.
4. Wait for the installation to complete. If prompted, restart your computer to finish the setup.
5. After restarting, open Docker Desktop from the Start menu. Accept the license agreement and allow Docker to start the Docker Engine.
6. Open Command Prompt or PowerShell and run the following command:
7. docker --version

If a version number is displayed, Docker has been installed successfully and is ready for use.

Docker

Docker is an open-source platform designed to simplify application development, deployment, and distribution. It packages an application together with all its dependencies, libraries, configuration files, and runtime environment into a single unit called a container. This ensures that the application behaves consistently across different environments, whether it is running on a developer's computer, a testing server, or a production system. Docker eliminates compatibility issues commonly described as "it works on my machine" by providing a standardized execution environment.

Docker Images

A Docker image is a lightweight, standalone, and read-only template used to create Docker containers. It contains everything required to run an application, including the application code, runtime environment, system libraries, tools, dependencies, and configuration files.

Docker images are built using multiple layers, where each layer represents a specific instruction such as installing software packages, copying application files, or configuring environment variables. Since images are immutable (read-only), they cannot be modified after they are created. Any changes require building a new image.

Docker Containers

A Docker container is the running instance of a Docker image. It provides an isolated environment in which an application executes independently of the host operating system and other containers.

When a container is created, Docker adds a thin writable layer on top of the read-only image layers. This writable layer allows the application to create files, update data, generate logs, and temporarily store information while the container is running. Containers provide process isolation and resource efficiency, ensuring applications run consistently across different systems while maintaining portability and reliability.

Docker Command:

1. docker –version

2. docker search ubuntu
3. docker search ubuntu --filter stars=100
4. docker search nginx
5. docker pull redis
6. docker run -d --name neerajdb redis

7. docker ps
8. docker inspect neerajdb
9. docker logs -f neerajdb
10. docker exec -it neerajdb sh
11. stop neerajdb
12. docker rm neerajdb docker rmi redis
13. docker images
14. docker pull nginx
15. docker run -d -p 8081:80 --name mynginx2 nginx
16. docker stop mynginx2

docker

Implementation:

1. docker --version

```
C:\Users\15L>docker --version
Docker version 29.2.1, build a5c7197
```

2. docker search ubuntu

```
C:\Users\15L>docker search ubuntu
NAME                              DESCRIPTION                                     STARS     OFFICIAL
ubuntu                            Ubuntu is a Debian-based Linux operating sys…   17859     [OK]
ubuntu/squid                      Squid is a caching proxy for the Web. Long-t…   129
ubuntu/nginx                      Nginx, a high-performance reverse proxy & we…   141
ubuntu/bind9                      BIND 9 is a very flexible, full-featured DNS…   120
ubuntu/kafka                      Apache Kafka, a distributed event streaming …   62
ubuntu/apache2                    Apache, a secure & extensible open-source HT…   104
ubuntu/zookeeper                  ZooKeeper maintains configuration informatio…   15
ubuntu/prometheus                 Prometheus is a systems and service monitori…   80
ubuntu/jre                        Chiseled Java runtime based on Ubuntu. Long-…   25
ubuntu/dotnet-aspnet              Chiseled Ubuntu runtime image for ASP.NET a…   26
ubuntu/python                     A chiselled Ubuntu rock with the Python runt…   33
ubuntu/mysql                      MySQL open source fast, stable, multi-thread…   74
ubuntu/postgres                   PostgreSQL is an open source object-relation…   45
ubuntu/redis                      Redis, an open source key-value store. Long-…   26
ubuntu/dotnet-runtime             Chiselled Ubuntu runtime image for .NET apps…   24
ubuntu/dotnet-deps                Chiselled Ubuntu for self-contained .NET & A…   16
ubuntu/grafana                    Grafana, a feature rich metrics dashboard & …   12
ubuntu/jdk                        Ubuntu rock for OpenJDK                         0
ubuntu/memcached                  Memcached, in-memory keyvalue store for smal…   7
ubuntu/rabbitmq                   Ubuntu-based RabbitMQ image - an open source…   0
ubuntu/mimir                      Ubuntu ROCK for Mimir, a horizontally scalab…   0
ubuntu/loki                       Grafana Loki, a log aggregation system like …   2
ubuntu/traefik                    Ubuntu ROCK for Traefik, a modern HTTP rever…   3
ubuntu/alertmanager               Ubuntu Rock for Alertmanager. It handles ale…   1
ubuntu/opentelemetry-collector    This is a rock for the OpenTelemetry Collect…   0

C:\Users\15L>
```

3. search ubuntu --filter stars=100

```
C:\Users\15L>docker search ubuntu --filter STARS=100
NAME              DESCRIPTION                                     STARS     OFFICIAL
ubuntu            Ubuntu is a Debian-based Linux operating sys…   17859     [OK]
ubuntu/squid      Squid is a caching proxy for the Web. Long-t…   129
ubuntu/nginx      Nginx, a high-performance reverse proxy & we…   141
ubuntu/bind9      BIND 9 is a very flexible, full-featured DNS…   120
ubuntu/apache2    Apache, a secure & extensible open-source HT…   104
```

4. docker search nginx

```
C:\Users\15L>docker search nginx
NAME                                      DESCRIPTION                                     STARS     OFFICIAL
nginx                                     Official build of Nginx.                        21340     [OK]
nginx/nginx-ingress                       NGINX and  NGINX Plus Ingress Controllers fo…   122
nginx/nginx-prometheus-exporter           NGINX Prometheus Exporter for NGINX and NGIN…   52
nginx/nginx-ingress-operator              NGINX Ingress Operator for NGINX and NGINX P…   4
nginx/nginxaas-loadbalancer-kubernetes                                                    1
bitnamicharts/nginx                       Bitnami Helm chart for NGINX Open Source        4
ubuntu/nginx                              Nginx, a high-performance reverse proxy & we…   141
kasmweb/nginx                             An Nginx image based off nginx:alpine and in…   9
rancher/nginx                                                                             4
linuxserver/nginx                         An Nginx container, brought to you by LinuxS…   236
dtagdevsec/nginx                          T-Pot Nginx                                     0
paketobuildpacks/nginx                                                                    0
vmware/nginx                                                                              3
gluufederation/nginx                       A customized NGINX image containing a consu…   1
cleanstart/nginx                          Secure by Design, Built for Speed, Hardened …   0
activestate/nginx                         ActiveState's customizable, low-to-no vulner…   0
antrea/nginx                              Nginx server used for Antrea e2e testing        0
intel/nginx                                                                               0
docksal/nginx                             Nginx service image for Docksal                 1
geokrety/nginx                            Our customized nginx image                      0
circleci/nginx                            This image is for internal use                  2
ilios/nginx                               Nginx customized to run Ilios along with the…   0
corpusops/nginx                           https://github.com/corpusops/docker-images/     1
dockette/nginx                            Nginx SSL / HSTS / HTTP2                        3
wayofdev/nginx                            Nginx Docker image for PHP development. SSL-…   0

C:\Users\15L>
```

5. docker pull redis

```
C:\Users\15L>docker pull redis
Using default tag: latest
latest: Pulling from library/redis
fd6b3803006b: Pull complete
234dd6735104: Pull complete
8fd80e87c6f1: Pull complete
6f0ec8fc933c: Pull complete
4f4fb700ef54: Pull complete
c1af3958d6f8: Pull complete
280ce3063e4e: Download complete
1bd39139fcfd: Download complete
Digest: sha256:234c902a2db49461a129e2d4aeff85b28cf20187ed274a67f6e50995fa713c7b
Status: Downloaded newer image for redis:latest
docker.io/library/redis:latest

C:\Users\15L>
```

6. docker run -d --name neerajdb redis

```
C:\Users\15L>docker run -d --name neerajdb redis
7dec7f70866eb45807a4495830246a066ee184d2caae476e02f5a298928c6b30

C:\Users\15L>
```

7. docker ps

```
C:\Users\15L>docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS          PORTS                                       NAMES
7dec7f70866e   redis     "docker-entrypoint.s…"   32 seconds ago   Up 28 seconds   6379/tcp                                    neerajdb
790bdd38d6a4   nginx     "/docker-entrypoint.…"   17 minutes ago   Up 17 minutes   0.0.0.0:8081->80/tcp, [::]:8081->80/tcp     mynginx2

C:\Users\15L>
```

8. docker inspect neerajdb

docker

```
C:\Users\15L>docker inspect neerajdb
[
    {
        "Id": "7dec7f70866eb45807a4495830246a066ee184d2caae476e02f5a298928c6b30",
        "Created": "2026-07-22T08:26:54.605378393Z",
        "Path": "docker-entrypoint.sh",
        "Args": [
            "redis-server"
        ],
        "State": {
            "Status": "running",
            "Running": true,
            "Paused": false,
            "Restarting": false,
            "OOMKilled": false,
            "Dead": false,
            "Pid": 841,
            "ExitCode": 0,
            "Error": "",
            "StartedAt": "2026-07-22T08:26:54.666055812Z",
            "FinishedAt": "0001-01-01T00:00:00Z"
        },
        "Image": "sha256:234c902a2db49461a129e2d4aeff85b28cf20187ed274a67f6e50995fa713c7b",
        "ResolvConfPath": "/var/lib/docker/containers/7dec7f70866eb45807a4495830246a066ee184d2caae476e02f5a298928c6b30/resolv.conf",
        "HostnamePath": "/var/lib/docker/containers/7dec7f70866eb45807a4495830246a066ee184d2caae476e02f5a298928c6b30/hostname",
        "HostsPath": "/var/lib/docker/containers/7dec7f70866eb45807a4495830246a066ee184d2caae476e02f5a298928c6b30/hosts",
        "LogPath": "/var/lib/docker/containers/7dec7f70866eb45807a4495830246a066ee184d2caae476e02f5a298928c6b30/7dec7f70866eb45807a4495830246a066ee184d2caae476e02f5a298928c6b30-json.log",
        "Name": "/neerajdb",
        "RestartCount": 0,
        "Driver": "overlayfs",
        "Platform": "linux",
        "MountLabel": "",
        "ProcessLabel": "",
        "AppArmorProfile": "",
        "ExecIDs": null,
        "HostConfig": {
            "Binds": null,
            "ContainerIDFile": "",
            "LogConfig": {
                "Type": "json-file",
                "Config": {}
            },
            "NetworkMode": "bridge",
            "PortBindings": {},
            "RestartPolicy": {
                "Name": "no",
```

9. docker logs -f neerajdb

```
C:\Users\15L>docker logs -f neerajdb
Starting Redis Server
1:C 22 Jul 2026 08:26:54.783 * oO0OoO0OoO0Oo Redis is starting oO0OoO0OoO0Oo
1:C 22 Jul 2026 08:26:54.783 * Redis version=8.8.0, bits=64, commit=00000000, modified=1, pid=1, just started
1:C 22 Jul 2026 08:26:54.783 * Configuration loaded
1:M 22 Jul 2026 08:26:54.783 * monotonic clock: POSIX clock_gettime
1:M 22 Jul 2026 08:26:54.784 * Running mode=standalone, port=6379.
1:M 22 Jul 2026 08:26:54.784 * <bf> RedisBloom version 8.8.0 (Git=unknown)
1:M 22 Jul 2026 08:26:54.784 * <bf> Registering configuration options: [
1:M 22 Jul 2026 08:26:54.784 * <bf>     { bf-error-rate       :      0.01 }
1:M 22 Jul 2026 08:26:54.784 * <bf>     { bf-initial-size     :       100 }
1:M 22 Jul 2026 08:26:54.784 * <bf>     { bf-expansion-factor :         2 }
1:M 22 Jul 2026 08:26:54.784 * <bf>     { cf-bucket-size      :         2 }
1:M 22 Jul 2026 08:26:54.784 * <bf>     { cf-initial-size     :      1024 }
1:M 22 Jul 2026 08:26:54.784 * <bf>     { cf-max-iterations   :        20 }
1:M 22 Jul 2026 08:26:54.784 * <bf>     { cf-expansion-factor :         1 }
1:M 22 Jul 2026 08:26:54.784 * <bf>     { cf-max-expansions   :        32 }
1:M 22 Jul 2026 08:26:54.784 * <bf> ]
1:M 22 Jul 2026 08:26:54.785 * Module 'bf' loaded from /usr/local/lib/redis/modules//redisbloom.so
1:M 22 Jul 2026 08:26:54.789 * <search> search-workers default: 16 (min of MAX_WORKER_THREADS=16 and CPU cores)
1:M 22 Jul 2026 08:26:54.789 * <search> Redis version found by RedisSearch : 8.8.0 - oss
1:M 22 Jul 2026 08:26:54.789 * <search> RediSearch version 8.8.0 (Git=d2ae026)
1:M 22 Jul 2026 08:26:54.789 * <search> Low level api version 1 initialized successfully
1:M 22 Jul 2026 08:26:54.789 * <search> gc: ON, prefix min length: 2, min word length to stem: 4, prefix max expansions: 200, query timeout (ms): 500, timeout policy: return, oom policy: return, cursor read si
ze: 1000, cursor max idle (ms): 300000, max doctable size: 1000000, max number of search results:  1000000, default scorer: BM25STD,
1:M 22 Jul 2026 08:26:54.789 * <search> Initialized thread pools!
1:M 22 Jul 2026 08:26:54.789 * <search> Enabled workers threadpool of size 16
1:M 22 Jul 2026 08:26:54.792 * <search> Subscribe to config changes
1:M 22 Jul 2026 08:26:54.792 * <search> Subscribe to cluster slot migration events
1:M 22 Jul 2026 08:26:54.792 * <search> Enabled role change notification
1:M 22 Jul 2026 08:26:54.792 * <search> Cluster configuration: AUTO partitions, type: 0, coordinator timeout: 0ms
1:M 22 Jul 2026 08:26:54.792 * Module 'search' loaded from /usr/local/lib/redis/modules//redisearch.so
1:M 22 Jul 2026 08:26:54.793 * <timeseries> RedisTimeSeries version 80800, git_sha=42ca4f1078fca732b7f9256adbf25914d67e1cc9
1:M 22 Jul 2026 08:26:54.793 * <timeseries> Redis version found by RedisTimeSeries : 8.8.0 - oss
1:M 22 Jul 2026 08:26:54.793 * <timeseries> Registering configuration options: [
1:M 22 Jul 2026 08:26:54.793 * <timeseries>     { ts-compaction-policy   :             }
1:M 22 Jul 2026 08:26:54.793 * <timeseries>     { ts-num-threads         :           3 }
1:M 22 Jul 2026 08:26:54.793 * <timeseries>     { ts-libmr-protocol      :    INTERNAL }
1:M 22 Jul 2026 08:26:54.793 * <timeseries>     { ts-retention-policy    :           0 }
1:M 22 Jul 2026 08:26:54.793 * <timeseries>     { ts-duplicate-policy    :       block }
1:M 22 Jul 2026 08:26:54.793 * <timeseries>     { ts-chunk-size-bytes    :        4096 }
1:M 22 Jul 2026 08:26:54.793 * <timeseries>     { ts-encoding            :  compressed }
1:M 22 Jul 2026 08:26:54.793 * <timeseries>     { ts-ignore-max-time-diff:           0 }
1:M 22 Jul 2026 08:26:54.793 * <timeseries>     { ts-ignore-max-val-diff :    0.000000 }
1:M 22 Jul 2026 08:26:54.793 * <timeseries> ]
1:M 22 Jul 2026 08:26:54.793 * <timeseries> Detected redis oss
1:M 22 Jul 2026 08:26:54.793 * <timeseries> Subscribe to ASM events
1:M 22 Jul 2026 08:26:54.793 * <timeseries> Enabled diskless replication
```

10. docker exec -it neerajdb sh

```
C:\Users\15L>docker exec -it neerajdb sh
# exit
```

11. stop neerajdb

```
C:\Users\15L>docker stop neerajdb
neerajdb

C:\Users\15L>
```

12. docker rm neerajdb docker rmi redis

```
C:\Users\15L>docker stop neerajdb
neerajdb

C:\Users\15L>docker rm neerajdb
neerajdb

C:\Users\15L>docker rmi redis
Untagged: redis:latest
Deleted: sha256:234c902a2db49461a129e2d4aeff85b28cf20187ed274a67f6e50995fa713c7b

C:\Users\15L>
```

docker

13. docker images

```
C:\Users\15L>docker images

IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
hello-world:latest   ef54e839ef54       25.9kB         9.52kB
nginx:latest         5a88c9c45479        241MB           66MB   U
ubuntu:latest        3131b4cc82a7        161MB         45.3MB   U

C:\Users\15L>
```

14. docker pull nginx

```
C:\Users\15L>docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
Digest: sha256:5a88c9c45479443d7be2eadc894b4ed0a9801bae03d97a5760ae13b5c2005942
Status: Image is up to date for nginx:latest
docker.io/library/nginx:latest

C:\Users\15L>
```

15. docker run -d -p 8081:80 --name mynginx2 nginx

```
C:\Users\15L>docker run -d -p 8080:80 --name mynginx nginx
docker: Error response from daemon: Conflict. The container name "/mynginx" is already in use by container "e41f9ed60c3ca35e6e09a3c23d07ee16fd966bcc8ae8c8b25600780af7a20da2". You have to remove (
t container to be able to reuse that name.

Run 'docker run --help' for more information

C:\Users\15L>docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS                      PORTS                  NAMES
ba6ad95ee1e4   ubuntu         "/bin/bash"              11 minutes ago   Exited (0) 10 minutes ago                          gracious_mirzakhani
07c20d772d7d   ubuntu         "--it"                   15 minutes ago   Created                                            pensive_haibt
f7b09cfcb56e   nginx          "/docker-entrypoint.…"   45 hours ago     Created                                            practice
e41f9ed60c3c   nginx          "/docker-entrypoint.…"   45 hours ago     Exited (255) 27 hours ago   0.0.0.0:8080->80/tcp   mynginx
506fbcc93880   redis:latest   "docker-entrypoint.s…"   4 months ago     Exited (0) 4 minutes ago                           distracted_babbage

C:\Users\15L>docker stop mynginx
mynginx

C:\Users\15L>docker rm mynginx
mynginx

C:\Users\15L>docker run -d -p 8080:80 --name mynginx nginx
2fda68e25078eadfffa279ac97bfb2e97684faf31ad190b716091db665501162
docker: Error response from daemon: ports are not available: exposing port TCP 0.0.0.0:8080 -> 127.0.0.1:0: listen tcp 0.0.0.0:8080: bind: Only one usage of each socket address (protocol/network
is normally permitted.

Run 'docker run --help' for more information

C:\Users\15L>netstat -ano | findstr :8080
  TCP    0.0.0.0:8080           0.0.0.0:0              LISTENING       5428
  TCP    [::]:8080              [::]:0                 LISTENING       5428

C:\Users\15L>tasklist /FI "PID eq 12345"
INFO: No tasks are running which match the specified criteria.

C:\Users\15L>docker run -d -p 8081:80 --name mynginx2 nginx
790bdd38d6a4ac9e30fa6eff5a27fe1b1f1ecc433217638d5c5849e58f75c83d
```

16. docker stop mynginx2

```
mynginx2

C:\Users\15L>docker rmi nginx
Error response from daemon: conflict: unable to delete nginx:latest (must be forced) - container 790bdd38d6a4 is using its referenced image 5

C:\Users\15L>docker rm nginx
Error response from daemon: No such container: nginx

C:\Users\15L>

C:\Users\15L>docker rm mynginx2
mynginx2

C:\Users\15L>
```

17. docker rmi nginx docker images

docker

```
C:\Users\15L>docker rmi nginx
Untagged: nginx:latest
Untagged: nginx@sha256:5a88c9c45479443d7be2eadc894b4ed0a9801bae03d97a5760ae13b5c2005942
Deleted: sha256:4e5db4761e0ff445f7fd29aad680ad28e8abf7d204895557f145d65535abcc1c
Deleted: sha256:2b3d10d90a77d4926b101279e7a65873c42b162087f614f75e88fa5075578458
Deleted: sha256:ce85d358b2fc8993b1f4258150e18ad68036164a2c1c5ec6cfe9f73b38c79038
Deleted: sha256:06f06fccf12160f5fb2d54c5810fee894e7171b9a3eb1dc07a9895af95f39fb4
Deleted: sha256:2d98d2a2e4c526409607174e7167b3961d4bd120ea1183772b248435b1d4ae7a
Deleted: sha256:4fad4b776a969a9a24fe73959a688c19525df696299519da94b5dd9224a74828
Deleted: sha256:135e056db1b5ffaad85c4e52f57983f61a937e246d79a5609cd5e042be60574a
Deleted: sha256:f2ec4de84f559f5c7be4233b589cdbdbb5507807e05621b77320edd55a1f2a0f

C:\Users\15L>docker images
REPOSITORY                 TAG                                       IMAGE ID       CREATED         SIZE
ubuntu                     latest                                    de7345b16e94   7 days ago      100MB
sample-flask-app           v1                                        f7d64ef4ebbb   3 months ago    141MB
myredis                    version1                                  5c08b3fad205   3 months ago    140MB
redis                      <none>                                    c234a24851ba   4 months ago    140MB
hello-world                latest                                    1b44b5a3e06a   11 months ago   10.1kB
node                       latest                                    8bbb9d1bc633   2 years ago     1.1GB
nginx                      1.0                                       5682b6dc087e   2 years ago     169MB
ubuntu                     <none>                                    ca2b0f26964c   2 years ago     77.9MB
ubuntu                     <none>                                    3db8720ecbf5   2 years ago     77.9MB
mysql                      latest                                    019814493c7a   2 years ago     632MB
httpd                      latest                                    2776f4da9d55   2 years ago     167MB
redis                      <none>                                    d1397258b209   2 years ago     138MB
docker/welcome-to-docker   latest                                    c1f619b6477e   2 years ago     18.6MB
getting-started            latest                                    8d8a2af2207f   3 years ago     265MB
docker/getting-started     latest                                    3e4394f6b72f   3 years ago     47MB
docker/desktop-git-helper  5a4fca126aadcd3f6cc3a011aa991de982ae7000   efe2d67c403b   4 years ago     44.2MB
```

Conclusion

In this experiment, the fundamental concepts of Docker and application containerization were explored through both theoretical study and practical implementation. The experiment introduced Docker as a platform for creating standardized and isolated application environments while highlighting the distinction between Docker images, which are read-only templates, and Docker containers, which are their active and executable instances.

During the practical session, the complete lifecycle of containers was demonstrated using standard Redis and Nginx images. The experiment successfully illustrated how to:

- Search for Docker images from a registry.
- Download (pull) images to the local system.
- Create and run containers in the background.
- Monitor, inspect, and access the shell of running containers.
- Stop and remove containers, followed by deleting the associated Docker images to clean up the environment.

Overall, this experiment provided practical experience in creating, deploying, managing, and maintaining containerized applications. It also demonstrated how Docker enables applications to run consistently and reliably across different computing environments, making software deployment more efficient and portable.